Patch-Aware KV Cache Compression for Byte-Level Language Models: Entropy Is Not a Good Keep Signal, Attention Is

Group 12
CS F429 Natural Language Processing, BITS Pilani Hyderabad
Part 3 report

Abstract

Byte-level language models read raw UTF-8, so a sentence spans about four times as many positions as in BPE tokens and the key-value (KV) cache grows four times faster. The Byte Latent Transformer (BLT) groups bytes into patches using the next-byte entropy of a small model, which suggested a free eviction signal: keep the high-entropy patches. We tested this on a 13.3M-parameter BLT-style model with three seeds per setting, paired bootstrap intervals, a held-out test split and random and recency controls. The proposal fails. Keeping high-entropy patches is worse than recency in 70 and worse than random in 68 of 72 comparisons, better in none, and the penalty grows about tenfold when the models are trained ten times longer. What works is different: a SnapKV-style rule scored on the model's own attention beats recency in 24 of 24 online comparisons; eviction plus 4-bit quantization keeps 7.4% of a WikiText cache for a cost of +0.0012 bits per byte; and a fixed-size recency or SnapKV-style cache follows documents four times longer than the training context, and 20 PG-19 books, better than resetting the context. We state plainly what is new, what failed, and what the small scale cannot show.

1 Introduction

Task. We study language modelling: predicting the next symbol of a text given the previous ones, scored by bits per byte (BPB) on held-out documents. Production systems use subword tokenizers such as BPE [15]. Byte-level models remove the tokenizer and its failure modes on rare words, typos, code and other scripts [13, 16, 18, 19], but a byte sequence is about four times longer than its BPE sequence (4.0 bytes per token in our data).

Why the cache matters. When a decoder generates text, it stores the keys and values of every earlier position in every layer. This KV cache, not the weights, limits batch size and context length at deployment. A byte-level Transformer of our width needs 151 MB per sequence at 16,384 bytes against 38 MB for BPE (Table 5, Figure 4). The BLT [13] reduces this by running its large global Transformer over *patches* of bytes, one cache entry per patch, and it already computes a next-byte entropy to decide where patches end.

Idea and outcome. Because that entropy is already computed, using it to decide which patches to keep or to store at low precision would cost nothing extra. Related work uses uncertainty to set cache budgets or as one term of a mixed score on subword models (Section 2); we found no test of entropy-ranked retention on byte-level models. We tested it and it is a reproducible negative result. The positive findings come from the controls and baselines we built to test it.

Contributions.

1. A BLT-style 13.3M-parameter byte model with entropy or fixed-stride patching, compared with a classical Kneser-Ney byte 4-gram and a 13.8M BPE Transformer, with three seeds per setting.
2. A negative result: keeping high-entropy patches is worse than random and recency in 68 to 70 of 72 comparisons and the penalty grows ten-fold with training length (Section 5.3).
3. What works: SnapKV-style attention scoring, eviction with 4-bit quantization (7.4% of the cache bytes for +0.0012 BPB), and bounded-memory streaming that beats context resets (Section 5.5).
4. An online, tested attention-policy implementation (global layers run patch by patch with a real cache), and an analytic compute count that shows where the BLT does and does not save (Section 5.5).
5. Quantitative and qualitative error analysis (Section 5.6) and an honest account of problems and limits (Sections 6 and 7).

2 Literature survey

Tokens and bytes. BPE merges frequent byte pairs into a fixed vocabulary [15]. Token-free models work on bytes directly: ByT5 [18] shows robustness at the cost of longer sequences; MegaByte [19] and SpaceByte [16] use multiscale or space-triggered blocks to shorten the expensive part. The Byte Latent Transformer [13] forms patches from the next-byte entropy of a small byte model, runs a large latent Transformer over patches, and reports that patches scale better than tokens. Our model is a simplified BLT: mean pooling instead of cross-attention, no n-gram hash embeddings, and a 13.3M size.

KV cache eviction. StreamingLLM keeps a few initial "sink" positions and a recent window so a model can stream far past its training length [17]. H2O keeps "heavy hitters", the positions with the largest accumulated attention [20]. SnapKV selects positions that the last queries of the prompt attend to [10]. All three act

on subword models. We implement recency with a sink, H2O and a SnapKV-style rule (attention from the last 16 patches) as policies over patch entries.

KV quantization. KIVI quantizes keys per channel and values per token to 2 bits without tuning [11]; KVQuant adds non-uniform and outlier-aware schemes for very long contexts [5]. We use a simpler symmetric per-vector absmax quantizer and ask only whether the entropy signal can allocate bits.

Uncertainty in cache management. CONF-KV sets the per-step cache budget from the next-token entropy and ranks tokens by attention and recency, with random eviction under the same schedule as a control [9]. InfoKV mixes predictive entropy and layer-wise representation change with attention scores [6]. Counter-causal surprise scores cached tokens by how well later tokens predict them [3]. A learned controller chooses 2 to 16 bits per token using features that include entropy [4]. Scratchpad Patching uses next-byte uncertainty to add temporary slots inside byte patches and reports a smaller KV cache [21]. We read CONF-KV and the counter-causal paper in full and only the abstracts of the other three, so our statement of difference from those three is provisional. In all of these entropy is one ingredient or sets a budget; none, to our knowledge, ranks which patches of a byte-level model to keep by entropy alone against random and recency at equal bytes, which is the question we test.

Classical baseline. Interpolated Kneser-Ney smoothing [1, 7] is the strongest classical n-gram method and gives a floor that any neural model should beat.

3 Dataset

Sources and fit. TinyStories [2] is a corpus of short stories written by a language model with a small vocabulary; it trains small models quickly and gives short documents (about 890 bytes). WikiText-103 [12] holds long, edited Wikipedia articles (about 18 KB each), which suits long-context cache experiments. Both have a standard, citable form and the two sit at opposite ends of difficulty (our baselines reach 0.69 and 1.52 BPB). The Stack [8] (the Smol sample) adds code, because a byte model should not fail on code. PG-19 [14] supplies very long documents. The first three are used for training and validation; the Wikipedia test split was evaluated once at the end.

Limitations and bias. TinyStories is synthetic, narrow and repetitive; long training overfits it (37 epochs of a 26.7 MB file, Section 5.4). WikiText-103 is English, encyclopedic and skewed to the topics Wikipedia editors cover; its Good and Featured article selection favours well-edited prose. The Stack sample is permissively licensed code with only exact duplicates removed, so near-duplicate files may make validation code slightly easy. PG-19 is old literary English, far from the WikiText-trained models, so its BPB is high. The release copy of WikiText train is cut to 3,000 articles to keep the demo download at 138 MB (our choice, not a GitHub limit); all reported runs use the full corpus.

Preprocessing. WikiText articles were rebuilt from the raw line export, which has no article boundaries, and placeholder tokens such as `@-@` were restored. All text is NFC normalised and exact duplicates are removed. Each split is written as one uint8 file of UTF-8 bytes with an offset array, so any window can be read at random and whole documents can be scored. Documents are isolated by a BOS byte and an attention mask. Code is split by the hash of its content so a file and its duplicate never fall on two sides. Every threshold and hyperparameter was tuned on validation only.

4 Method

Model. The model is a BLT-style network of 13,333,632 parameters. A local encoder (1 Transformer layer, width 256, 4 heads, rotary positions, causal window of 256 bytes) reads bytes. The encoder states of the bytes in a patch are mean-pooled and projected to width 384. A global Transformer (6 layers, width 384, 6 heads) runs causally over patches. A local decoder (2 layers, width 256, window 256) predicts each byte from its own encoder state and the global output of the patches that precede it. The global K and V tensors are the cache we compress: one entry per patch, $2 \cdot 6 \cdot 384 \cdot 2 = 9{,}216$ bytes at fp16. Step-by-step decoding equals the full forward pass to 10^{-5} (tested).

Patching. Entropy patching follows the BLT: a separate 3.3M-parameter byte model (4 layers, width 256, window 512) gives the next-byte entropy H_i, and a patch starts where H_i exceeds a threshold tuned on validation to give 4.5 bytes per patch (1.09 nats on TinyStories, 2.17 on WikiText). The decision uses only earlier bytes (tested). Fixed-stride patching (4 or 5 bytes) is the control. On the entropy patches 67.8% (TinyStories) and 66.7% (WikiText) of patch starts fall on word starts against 19.9% and 16.3% by chance.

Cache policies. A policy keeps at most B patch entries; the first patch (sink) and the 8 newest are always kept, and every policy chooses among the rest, so differences come from the signal and not from the window. Scores per old patch are:

- *recency*: newest first (StreamingLLM-style);
- *random*: three draws averaged;
- *keep high entropy* (the proposal): the mean entropy of the patch, $s_j = |P_j|^{-1} \sum_{i \in P_j} H_i$, keep the largest;
- *keep low entropy*: the opposite control;
- *H2O*: attention received so far, summed over queries, heads and layers;
- *SnapKV-style*: attention received from the last 16 patches only.

Entropy and recency policies are simulated by masks in one pass and tested equal to step-by-step decoding with entries really removed. H2O and SnapKV-style need the evicted cache, so `forward_online` runs the global layers patch by patch, chooses the entry to drop before the new patch is attended to, and accumulates attention as it is produced (tested for causality and for equality with the full cache at a large budget).

Table 1. Corpora. Bytes are after normalisation and deduplication. Evaluation sets are fixed and shared by every model. Counts for TinyStories and WikiText-103 are from `data/processed/*/stats.json` in the Part 2 release.

Corpus	Domain	Size and use
TinyStories	Synthetic children's stories	30,000 train stories (26.7 MB); 21,990 validation; evaluation on the first 500 validation stories (405,231 bytes); no test split
WikiText-103	English Wikipedia articles	29,002 train articles (526.4 MB, 442 duplicates removed); validation 60 articles (1,129,194 bytes); test 62 articles (1,269,744 bytes)
The Stack Smol	Python, Java, JavaScript files	28,229 files, 15% of each training batch in the code-mix models; 200 validation files
PG-19	Pre-1919 books	test split, 100 books (40.5 MB); first 100,000 bytes of 20 books, streaming only

Quantization. Each K and V vector is stored with symmetric absmax scaling at b bits plus one fp16 scale per head. Bytes per entry are 9,216 (fp16), 2,448 (4-bit, 27%) and 1,296 (2-bit, 14%). Tier policies give a fraction of patches a high width (4 bits) and the rest a low one (2 bits), chosen by recency, random, or entropy, against uniform widths. Eviction and quantization combine by quantizing every kept entry.

Streaming. For long documents the fixed-size cache runs 16,384-byte chunks (and, for PG-19, 100,000 bytes one byte at a time with `BLT.step`) against a baseline that resets the context every 4,096 bytes, the training length.

Compute. Section 5.5 counts multiply-accumulates analytically (formulas in `results/part3/flops.md`).

5 Experiments

5.1 Setup, metrics and baselines

Training. The main models train for 3,000 AdamW steps (batch 8 of 4,096 bytes, learning rate 10^{-3}, about 98M bytes seen, bf16). Some WikiText code-mix models train for 30,000 steps (about 983M bytes, 1.9 epochs) and at 16,384-byte context. Seed 0 of the prose-only models was trained on a laptop RTX 4060, the other runs on a cluster GPU; the same checkpoint scores identically on both.

Metric. Bits per byte is the total negative log2-probability of the document divided by its byte count. It does not depend on the tokenizer, so the 4-gram, the BPE Transformer and the byte models are compared on identical text. Its limits: it measures prediction, not downstream use such as retrieval; differences between cache policies are small (at most 0.03 BPB), so every comparison is paired by document and reported with a 95% bootstrap interval over documents (2,000 resamples); and the Part 2 BPE number is 0.4 to 0.5% optimistic because the first token of each 256-token chunk is context only while its bytes are counted (corrected values 0.6978 and 1.5257). Cache size is the mean resident global KV size computed from entry counts and bit widths, and measured on a GPU only for recency (Figure 4).

Baselines. Classical: interpolated Kneser-Ney byte 4-gram. Modern: a 13.8M BPE Transformer (6 layers, 6 heads, width 384, 8k vocabulary, 256-token context, 3,000 steps). Cache baselines: full cache, recency with a sink, random, H2O, SnapKV-style, and uniform 8, 4, 3 and 2 bits. The Transformer was trained before gradient clipping was added (clipping gives 0.6826 on TinyStories), so the modern baseline is slightly weak.

Table 2. BPB on identical documents, lower is better. TS is TinyStories, WT is WikiText-103. First rows are the Part 2 baselines. BLT rows are seed 0, 3,000 steps, validation; the last row is the mean of the three 30,000-step code-mix seeds (1.2949, 1.2983, 1.3118), which saw ten times more training than the baselines.

Model	TS val	WT val	WT test
KN byte 4-gram	1.8085	2.4103	2.4381
BPE Transformer 13.8M	0.6939	1.5200	1.5309
Entropy model 3.3M	0.8438	1.6983	n/a
BLT, stride 4	0.7726	1.7600	1.7894
BLT, entropy patching	0.7980	1.6801	1.6986
BLT, entropy, 30k steps	n/a	1.2983	n/a

5.2 Baselines and base models

Table 2 gives the comparison. Every neural model is far ahead of the 4-gram (a gap of 1.0 to 1.1 BPB on TinyStories and 0.65 to 0.9 on WikiText). At equal training the BPE Transformer is better than our BLT-style model on both sets (0.69 against 0.80 on TinyStories, 1.52 against 1.68 on WikiText). This is a smaller, shorter-trained model without the hashed n-gram embeddings or cross-attention of the original BLT, so we do not claim a win over BPE at equal training. With ten times more training (a comparison that favours the BLT, since the Transformer kept 3,000 steps) the WikiText code-mix BLT reaches 1.29 to 1.31.

Entropy patching against fixed stride, three seeds each (mean full-cache BPB): on WikiText 1.6958 (entropy) against 1.7347 (stride 4) and 1.7629 (stride 5), with entropy patching lower in all three same-seed pairs and with fewer cache entries per byte than stride 4; on TinyStories 0.8183 against 0.8237 and 0.8521. Seed ranges are 0.056 and 0.028 for entropy patching and 0.135 and 0.044 for stride 4, so the WikiText gap of 0.039 is about the size of a seed range. We call it likely, not established. On TinyStories entropy patching ties stride 4. An earlier single-seed claim that entropy patching is 3.3% worse on TinyStories was withdrawn.

Table 3. BPB of the seed-0 entropy-patched model under cache budgets (first patch and 8 newest protected). Full cache: 0.7980 (TinyStories), 1.6801 (WikiText). Share of full cache bytes in brackets.

Budget	recency	random	low ent.	high ent.
TinyStories 32 (32%)	0.8078	0.8115	0.8089	0.8231
TinyStories 64 (56%)	0.8016	0.8033	0.8026	0.8109
WikiText 64 (15%)	1.6836	1.6860	1.6901	1.6922
WikiText 128 (28%)	1.6814	1.6833	1.6864	1.6885

5.3 Entropy-guided eviction: a negative result

Table 3 shows the proposal losing to random and to recency at every budget. The pattern holds across the evidence we collected:

- Over 72 comparisons (9 models in each of 4 settings, 2 budgets) keeping high entropy is worse than recency with the interval above zero in 70 and indistinguishable in 2, worse than random in 68 and indistinguishable in 4, and never better than either. The paired difference against random is +0.0003 to +0.0245 BPB.
- It holds in every seed, for fixed-stride models (so it is not caused by uneven entropy patches), with the code mix, on the WikiText test split (all 36 model and budget cells worse than random and recency), at 16,384 bytes (+0.016 to +0.021 against random at 64 to 512 patches), and for a recency-protected variant (newest half protected, other half by entropy: worse than recency in 23 of 24 cells).
- Ablations do not rescue it (three seeds): the model's own entropy instead of the separate entropy model, the first-byte score, the mean score and sink sizes 0 and 4 are worse than random in all seeds on both datasets; the maximum rule is worse in 3 of 3 (WikiText) and 2 of 3 (TinyStories) seeds; only the sum rule on TinyStories is ever better than random (one seed, −0.0024).
- Keeping *low* entropy beats keeping high entropy in 42 of 72 comparisons, is worse in 19 and ties in 11, so low-entropy retention is not a dependable alternative either.
- A byte-level cache (one entry per byte) loses much more than a patch cache at equal entry counts: at 32 entries recency costs +0.2286 BPB against +0.0133 with patches. Patches win per byte of cache; bytes win only with about 4.5 times more cache.

5.4 More training makes the penalty larger

A natural objection is that a small, short-trained model reads little from its cache. We trained WikiText code-mix models for 30,000 steps. They score 1.2949, 1.2983 and 1.3118 BPB and keeping high entropy costs +0.0287, +0.0284 and +0.0238 against random at 64 patches (against +0.0020 at 3,000 steps), with every interval excluding zero (Figure 1). Any eviction also costs more (recency at 64 patches costs 0.018 BPB over the full cache, from 0.002), and the fixed-stride model shows the same sign. The same experiment on TinyStories overfits (training loss 0.14 nats, validation BPB 1.6493 against 0.7980) and we draw no conclusion from it. So a small model is not what made the entropy rule look bad. A longer context does not rescue it either: our first 16,384-byte models were under-trained and suggested the penalty fades, a conclusion withdrawn after tuning (Section 6).

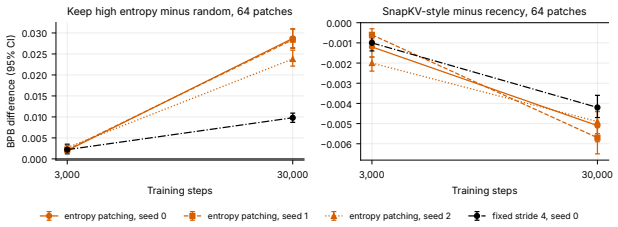


Figure 1. Ten times more training (3,000 to 30,000 steps, WikiText with code mix) makes the entropy penalty about ten times larger (left) and the SnapKV-style gain several times larger (right). Bars are 95% bootstrap intervals; three seeds of entropy patching, one of fixed stride.

5.5 What works: attention, quantization, bounded memory

Attention-based scoring. Online, the SnapKV-style rule is better than recency in all 24 comparisons on the 12 entropy-patched models (by 0.0002 to 0.0077 BPB, for example −0.0013 [−0.0018, −0.0009] on WikiText code mix, seed 0, 64 patches), and never worse than its offline approximation. With offline scoring it is also better than recency in 33 of 36 test-split cells (the other three tie) and, for the 30,000-step models, by 0.0051, 0.0057 and 0.0049 at 64 patches. Online H2O is worse than recency in 22 of 24: accumulated attention favours old patches that were popular early. The effect is small (for seed 0 on TinyStories at 32 patches recency costs 0.0098 over the full cache and SnapKV-style recovers 0.0041), but it is the one signal that beat recency, and it is attention, not surprise.

Quantization and combination. Uniform 4-bit K and V costs 0.0003 to 0.0014 BPB over fp16 across ten models at 27% of the bytes (uniform 3-bit 0.0014 to 0.0077, 2-bit 0.022 to 0.111). Entropy-guided precision tiers lose to random and to recency tiers in all ten models. Eviction and 4-bit together, mean of three seeds, are in Table 4: a WikiText cache of 128 patches in 4-bit holds 7.4% of the full cache bytes and costs +0.0012 BPB with SnapKV-style eviction (+0.0019 with recency), while keeping high entropy costs +0.0151. Costs of eviction and quantization add roughly linearly. No setting that keeps high-entropy patches is on the bytes-against-BPB frontier (Figure 2).

Bounded-memory streaming. Three WikiText code-mix models trained on 4,096-byte chunks decoded 46 validation documents longer than 8,192 bytes with a fixed patch budget. Recency with 64 patches beats the chunked full-cache baseline in all three seeds (by 0.0030, 0.0032 and 0.0008 BPB; with 256 patches by 0.0051,

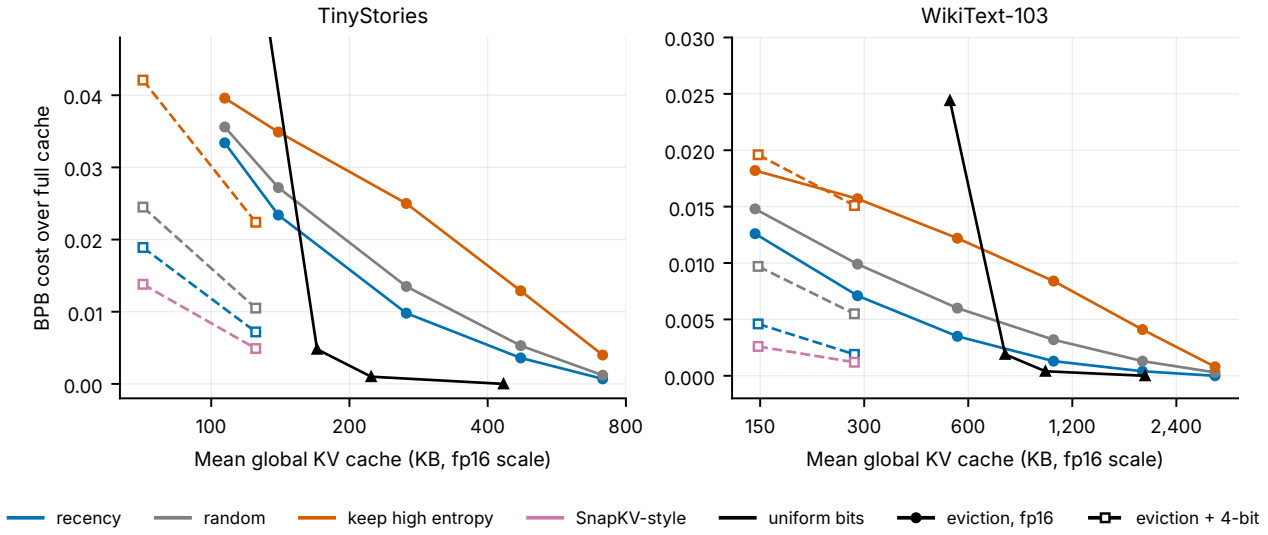


Figure 2. Cost over the full cache against cache size (seed-0 eviction and uniform rows from `comparison.md`; 4-bit rows are the three-seed mean from `deploy_combo.md`). Keeping high entropy (orange) is the worst eviction rule at every size and never on the frontier; SnapKV-style with 4-bit (pink, hollow) is the best low-memory point. Uniform 2 bits is off scale.

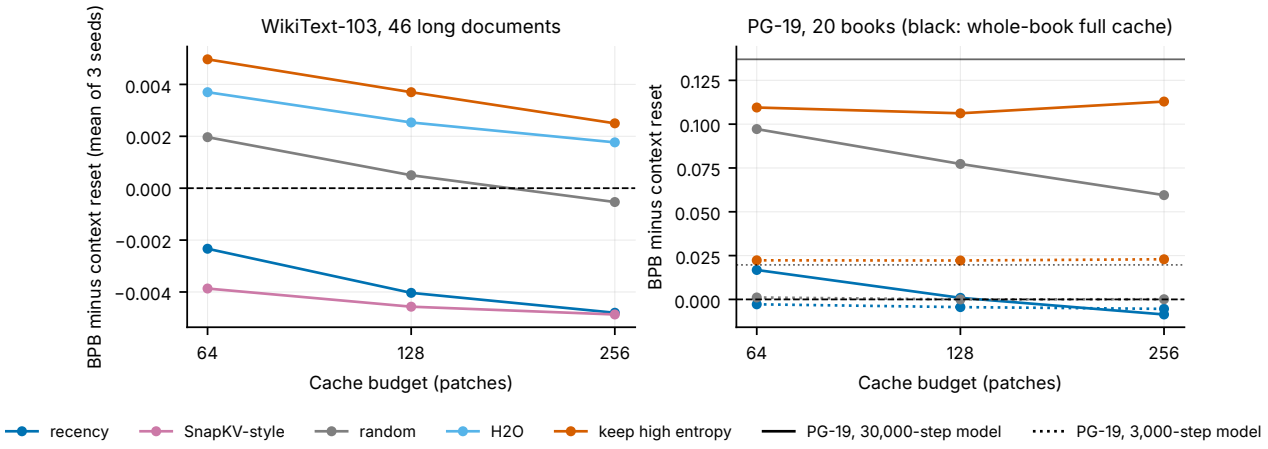


Figure 3. Bounded-memory streaming. Left: WikiText documents longer than 8,192 bytes decoded in 16,384-byte chunks (patch positions four times beyond training); below zero beats resetting the context every 4,096 bytes. Right: 20 PG-19 books of 100,000 bytes decoded one byte at a time (positions about 22,000). Recency is the only policy that is safe on books; keeping high entropy is worst but is confounded by distance (Section 5.5).

Table 4. Eviction with uniform 4-bit K and V, mean of three seeds, cost over the full fp16 cache in BPB (share of full cache bytes). Full cache: 0.8183 (TinyStories), 1.6958 (WikiText).

Setting	TinyStories	WikiText
SnapKV-style, 64, 4-bit	+0.0049 (15%)	+0.0026 (3.9%)
recency, 64, 4-bit	+0.0072 (15%)	+0.0046 (3.9%)
keep high ent., 64, 4-bit	+0.0224 (15%)	+0.0196 (3.9%)
SnapKV-style, 128, 4-bit	n/a	+0.0012 (7.4%)
recency, 128, 4-bit	n/a	+0.0019 (7.4%)

0.0058 and 0.0035), because the reset loses the context at every chunk boundary and a small continuous window does not. SnapKV-style is better still at 64 patches (0.0014, 0.0008 and 0.0024 below recency). Random, H2O and keeping high entropy are worse than recency at every budget in all seeds (Figure 3, left). On PG-19, with the 30,000-step model, 256 patches of recency beat the reset baseline by 0.0086 BPB [0.0079, 0.0093] and the whole-book full cache by 0.145; the whole-book full cache is worse than the reset baseline (by 0.137), so the model does not use a context 24 times longer than it was trained on. Keeping high-entropy patches is +0.093 to +0.122 worse than recency. Keeping old patches means attending at distances never seen in training, and we did not renumber positions inside the cache as StreamingLLM does, so PG-19 cannot separate a poor entropy signal from poor long-distance attention.

Memory and speed. Figure 4 gives the measured savings. The fixed 786 KB byte-window caches and the entropy model (about 2.1 MB of window cache and 3.3M parameters) are not reduced, so at 64 patches they exceed the global cache. At full precision the patch cache is about 0.9 times the size of a BPE Transformer cache of the same width, and a byte-level Transformer

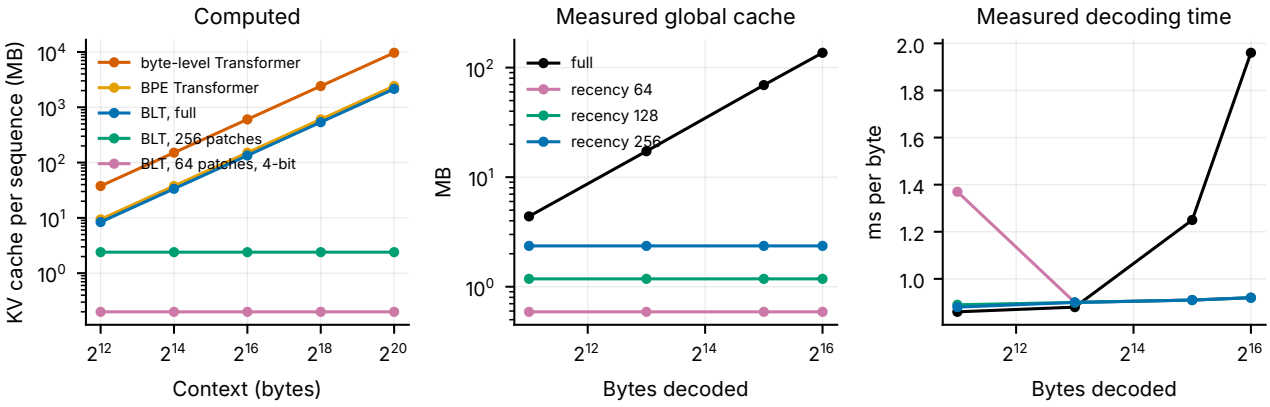


Figure 4. Cache memory. Left: computed KV size per sequence. Middle and right: measured on a GPU, one stream, real eviction of recency. A bounded cache holds the global cache at 0.59 to 2.36 MB while a full cache reaches 136.5 MB at 65,536 bytes; time per byte stays at 0.9 ms against a rise to 2.0 ms. The first recency point includes warm-up.

Table 5. Analytic MFLOPs per generated byte and KV entries (fp16 size in MB) at two contexts (`results/part3/flops.md`). † The BPE Transformer was trained for 256 tokens (about 1,024 bytes), so the 4,096-byte row is hypothetical.

	1,024 bytes		4,096 bytes	
Model	MFLOPs	MB	MFLOPs	MB
BLT, full cache	11.06	2.1	12.46	8.4
BLT, 64 patches	10.73	0.6	10.73	0.6
BLT, full, + ent. model	19.58	2.1	20.98	8.4
BLT, 64 patches, + ent. model	19.25	0.6	19.25	0.6
BPE Transformer	7.43	2.4	9.20†	9.4
Byte Transf., full	30.87	9.4	59.18	37.7
Byte Transf., 64 entries	22.02	0.6	22.02	0.6

needs about four times more.

Compute per byte. Table 5 is an analytic count, not a measurement. The BLT is cheaper than a byte-level Transformer (11.1 against 30.9 MFLOPs per byte at a context of 1,024 bytes), but it is *more* expensive than the BPE Transformer (7.4), because its local encoder and decoder run on every byte, and it needs the separate entropy model (8.5 MFLOPs per byte) when patches come from entropy; with it the total is 19.6. A 64-patch cache trims only the global attention term (0.3 MFLOPs at 1,024 bytes, 1.7 at 4,096). So the case for the BLT here is cache memory, not FLOPs.

Retrieval. In a planted-name test (a random four-syllable name repeated after 400 to 3,000 filler bytes) the models score the same with and without the earlier mention, even at 30,000 steps. No policy can be shown to help, and nothing about retrieval can be concluded at this scale. The entropy rule does keep the planted name's patches more often than random does (45% against 31% at 64 patches and 400 bytes), but the model gains nothing from them.

5.6 Error analysis

We scored 100 TinyStories documents (74,760 bytes, budget 32 patches) and 30 WikiText documents (122,880 bytes, 64 patches) with the seed-0 models on a CPU (`docs/error_analysis_part3.md`, generated by `scripts/error_analysis_part3.py`). The documents are the first ones that pass a length filter; the tables describe one seed and are not tests.

Quantitative. Table 6 splits the extra loss by the entropy tercile of the patch holding the byte. Every rule loses most on the high-entropy third, where the full-cache loss is also highest, but keeping high-entropy patches does not protect those bytes (+0.0392 against +0.0225 for recency on TinyStories; +0.0190 against +0.0062 on WikiText), and it also costs the easy bytes (+0.0194 against +0.0080 and +0.0152 against +0.0023 on the lowest third, which holds 57% and 55% of the bytes). The entropy rule keeps almost only high-entropy patches (44.6% and 18.1% of the highest third survive, none of the lowest) and they are short: the mean kept patch is 1.15 and 1.13 bytes long and 98.0% and 99.7% of them have one or two bytes, against 4.46 and 4.47 bytes for recency. This is a length confound (a surprising byte often forms its own patch, so a mean over a short patch is high). It does not explain the whole result, since the sum rule, which favours long patches, ties random on TinyStories and loses on WikiText (Section 5.3). By byte class, the entropy rule loses most on word-initial lowercase letters (+0.0437 against recency on WikiText) and on capitals in TinyStories (+0.1426), and gains a little on digits and capitals in WikiText (−0.0199 and −0.0083). The damage is concentrated: the largest 1% of bytes carry 30.4% (TinyStories) and 18.0% (WikiText) of the extra loss against recency, and entropy is worse than recency on 40.7% and 51.0% of bytes and better on 33.1% and 42.4%.

Why? Attention. Figure 5 shows the Spearman correlation between the entropy of an old patch (at least 8 patches back) and the attention it receives. It is negative in the middle and upper layers of most models: in all six layers of all three TinyStories seeds (−0.04 to −0.55), and in layers 2 to 5 of the three prose-only WikiText seeds (−0.007 to −0.39). In the seed-0 TinyStories model the highest-entropy third of old patches receives about half the attention of the lowest

Table 6. Extra loss over the full cache (BPB) by entropy tercile of the patch holding the byte. Share of bytes in brackets.

Set	Tercile	recency	high ent.	low ent.
TinyStories	lowest (57%)	+0.0080	+0.0194	+0.0065
	middle (32%)	+0.0104	+0.0272	+0.0130
	highest (11%)	+0.0225	+0.0392	+0.0253
WikiText	lowest (55%)	+0.0023	+0.0152	+0.0035
	middle (31%)	+0.0037	+0.0146	+0.0165
	highest (14%)	+0.0062	+0.0190	+0.0274

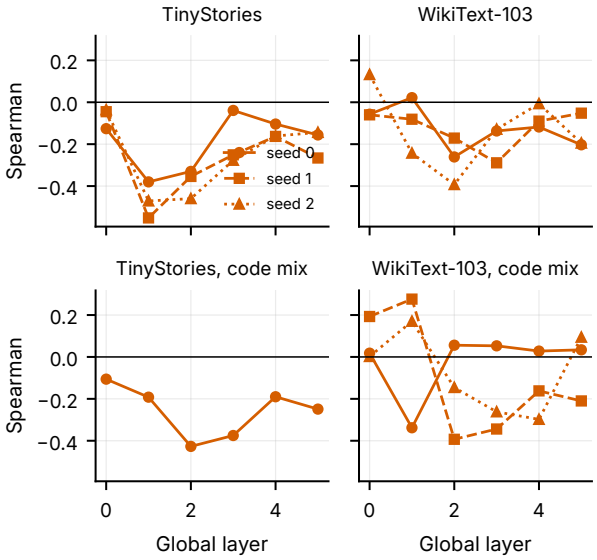


Figure 5. Spearman correlation between patch entropy and the attention received by old patches, per global layer and seed. It is mostly negative in the middle and upper layers: the model reads predictable patches more than surprising ones.

third in layers 1 and 2. The model reads predictable patches more than surprising ones, and surprising bytes are handled by the local windows. This is a correlation, not an intervention, and the WikiText code-mix seed 0 model shows the opposite sign in most layers while its eviction result has the same sign, so it is not a complete account.

Qualitative. In the examples `|` marks a patch boundary; all are real output.

Example 1, TinyStories document 0 (1,201 bytes, 4.0 bytes per patch). Boundaries fall before surprising letters and split names: `R|o|xy. |Ro|x|y |loved to |c|limb.`, while predictable words form whole patches (`She |had |never |seen |anything |`). At the last patch the caches (budget 32) hold, for recency, `played |together |all the |time. |And |Ro|x|y |l|earned that |...` (BPB 0.744, +0.010 over full); for keep high entropy, `|s|f| |B|s|s|Ro| |b|f|b|f|a|s|a|i|...` (0.751, +0.017): a string of one-letter fragments that keeps `Ro` but not `Roxy`; for keep low entropy, `Once upon a time|, in a |forest, there li|...|never |seen |ried |again, |...` (0.743, +0.009), whole words from the start of the story.

Example 2, WikiText document 0 (4,096 bytes, budget 64). The entropy cache at the last patch is `o|Ho|E|A|O|S|A|a|6| |a |e|5 |1|e|e|1|...`: capitals, digits and vowels, no words. BPB is 1.752 (+0.033) against 1.725 for recency (+0.006), whose cache holds the section on distribution (`Norway |to |the |A|zores |and |Mo|r|occo`).

Example 3, a locally predictable byte. In TinyStories document 56 the byte `d` in `They argue[d]` costs 0.02 bits with the full cache, 0.01 with recency and 5.97 with keep high entropy. The recent bytes are present (the newest 8 patches are protected), so what changes is the older cache; we hypothesise that a cache of fragments distorts the global state that feeds the decoder even for easy bytes, but we did not test it. A WikiText case: `to the northeast , i[s]` costs 0.84, 0.86 and 4.15 bits.

Example 4, where entropy helps. In TinyStories document 13 the byte `S` in `All the animals cheered for [S]am` costs 4.04 bits with the full cache, 7.35 with recency and 3.46 with keep high entropy. The name Sam appears several times earlier in the story. We did not check whether the entropy cache held a patch with the name. Such cases exist but are outweighed on average.

6 Challenges and how we handled them

No separate Part 1 problems were recorded; the list below covers the problems of Parts 2 and 3 (full entries in `CHALLENGES.md`).

- **Optimistic baseline scoring.** The Part 2 BPB counted the first token of each chunk, which is context only, making it 0.4 to 0.5% too low. We kept the reported numbers for traceability, disclosed the correction (0.6978, 1.5257) and score every byte in Part 3.
- **Seed variance as large as the effects.** A first single-seed result said entropy patching is 3.3% worse than fixed stride on TinyStories; seed spread is 0.056 to 0.135 BPB. We withdrew it, ran three seeds of every scheme and a stride-5 control, and paired every comparison by document.
- **Cache accounting bug.** The first policy evaluation counted full-cache entries twice, so byte shares were half the true shares. It surfaced when eviction and quantization rows were put in one table and the full-cache sizes disagreed (1,669 KB against 839 KB). Fixed, evaluations rerun, all tables regenerated; BPB was unaffected.
- **A wrong conclusion from under-trained models.** The first 16,384-byte models scored 1.89 BPB, worse than the 4k models, and suggested the entropy penalty fades with context. Tuning learning rate and batch size on validation gave 1.5843 and showed the penalty does not fade; the claim is withdrawn. The tuned models also saw four times more bytes, so context length and training amount are not separated.
- **Overfitting at ten times the training.** 30,000

steps on TinyStories are 37 epochs. We kept the WikiText scale-up as the evidence and listed the TinyStories rows only for completeness.

- **Code is gated and prose models cannot read it.** Models trained on prose score 7 to 8 BPB on code. A login by a team member built the code set; we added a 15% code mix to training, which costs about 0.05 BPB on WikiText and 0.09 on TinyStories and leaves the eviction conclusion unchanged.
- **A silent failure.** A CUDA out-of-memory error left a partly written entropy cache on disk while the queue carried on; reading the log caught it before training used zeros. The next queue now stops if the cache script fails.
- **Offline attention policies.** Mask simulation cannot see the evicted cache, so the first H2O and SnapKV-style scores were approximations. We built the online `forward_online` and tested it; the ranking held.
- **Models that do not retrieve.** The needle test is uninformative; we report retention rates and conclude nothing about retrieval.
- **PG-19 loading.** The standard dataset ships as a loading script that current libraries no longer run; we used a community mirror of the same test split and recorded the substitution.
- **Cluster limits.** One job at a time with a 2-day limit. Runs checkpoint, handle a signal before the time limit, requeue and resume; the path was verified end to end under one job id (signal at step 1,864, resume, finish at step 24,000). Three earlier attempts to show it failed for avoidable reasons that the docs record.
- **Reporting a negative result.** We tested the escape routes (other scores, own entropy, sink size, byte-level cache, more training, longer context, a protected recency window) before concluding, and added the attention rule that does beat recency.

7 Limitations

- Models are small (13.3M parameters, mostly 3,000 steps). The 30,000-step models show the same sign with larger effects, but a much larger model, or a task that needs retrieval, could behave differently. The needle test shows our models do not retrieve.
- TinyStories has no usable long-training result and no test split.
- Cache sizes are computed from entry counts; only recency eviction was measured on a GPU, and quantization is simulated by rounding.
- The 30,000-step fixed-stride model and the 16k models have one seed; the streaming runs use three seeds of one setting; PG-19 uses 20 books and out-of-domain models, and H2O and SnapKV-style were not run there because the step-decoding path has no attention policies.
- Seed 0 of the prose-only models was trained on a different GPU from the other seeds.
- The attention account of why entropy fails is a correlation; no intervention tests it, and it does not hold in every layer of every model.
- A model that fits TinyStories memorises it, and no result from it should be read as general.

8 Conclusion

What is new: a test of entropy-ranked cache retention on a byte-level patch model, against random and recency at equal bytes, with seeds, paired intervals and a test split; an online, tested attention-policy implementation on patch caches; and measured evidence that a fixed recency or SnapKV-style cache can stream documents several times longer than the training context better than resetting it. What failed: the central claim. The entropy that places patch boundaries is not a good signal for which patches to keep, and more training makes it worse. What is practical: 4-bit K and V plus a SnapKV-style or recency window keeps 7.4% of the WikiText cache bytes for +0.0012 to +0.0019 BPB. What we cannot say: whether the result holds for models orders of magnitude larger, or for tasks that need retrieval, because our models do not retrieve. Everything regenerates from the commands in the repository README; the figure script is `report/make_figures.py`.

References

[1] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13(4):359–394, 1999.

[2] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[3] Stephen Gould and Anton van den Hengel. Back from the future: Key-value cache management by counter-causal surprise. *arXiv preprint arXiv:2607.27600*, 2026.

[4] Sayed Pedram Haeri Boroujeni, Niloufar Mehrabi, Patrick Woods, Gabriel Hillesheim, and Abolfazl Razi. Don't waste bits! adaptive KV-cache quantization for lightweight on-device LLMs. *arXiv preprint arXiv:2604.04722*, 2026.

[5] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079.

[6] Jushi Kai, Zhuiri Xiao, Alexandra Birch, and Zhouhan Lin. Information-aware KV cache compression for long reasoning. *arXiv preprint arXiv:2606.26875*, 2026.

[7] Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[8] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[9] Yubo Li and Yidi Miao. CONF-KV: Confidence-aware

KV cache eviction with mixed-precision storage for long-horizon LLM. *arXiv preprint arXiv:2605.24786*, 2026.

[10] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[12] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[13] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[14] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.

[15] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. doi: 10.18653/v1/P16-1162.

[16] Kevin Slagle. SpaceByte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

[17] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[18] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[19] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. *arXiv preprint arXiv:2305.07185*, 2023.

[20] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2306.14048.

[21] Lin Zheng, Vasilisa Bashlovkina, Timothy Dozat, Dan Garrette, Laura Rimell, and Joshua Maynez. Scratchpad patching: Decoupling compute from patch size in byte-level language models. *arXiv preprint arXiv:2605.09630*, 2026.